

Fully Connected Networks

Siva Sai Nagender Vasireddy

1 Introduction

In this document, we discuss the mechanism of training a fully connected network on the classification task.

2 Task Description

Consider the task of classifying a vector $X \in \mathbb{R}^d$ into one of n classes $C \in [C_1, C_2, \dots, C_n]$. Let $f(x)$ be a two-layered fully connected network as shown in Fig. 1. The training mechanism of this network on the classification task is described in Section 3.

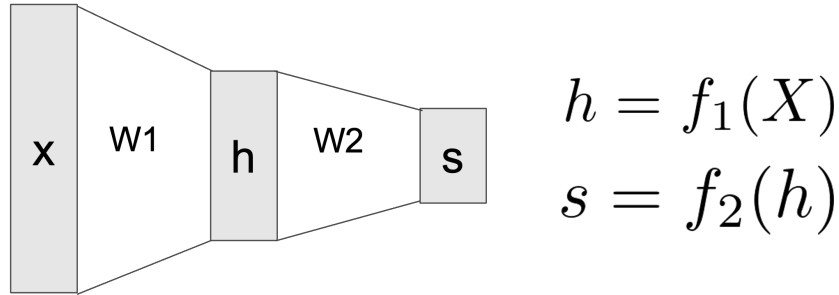


Figure 1: Image Credit: Slides from Dr. Yapeng Tian's Computer Vision class. Representation of a two-layer fully connected network. W_1 and W_2 are the transformation matrices of the first and second matrices respectively. $X \in \mathbb{R}^d$ is the input to the network. X is a one dimensional vector of size d . $h \in \mathbb{R}^{d_h}$ is the output of the first layer of the network which is also called the hidden layer. h is a one dimensional vector of size d_h . $s \in \mathbb{R}^n$ is the output of the second layer of the network which is also called the output layer. s is a one dimensional vector of size n . n is also the number of classes in the classification task.

3 Training Fully Connected Networks

We train neural networks over multiple iterations until the parameters converge. In each iteration, we select a random batch of samples that are representative of the dataset as described in Section 3.1. The next step is the forward pass in which the input is passed through the network to generate the output as described in Section 3.2. The output is then compared with the groundtruth labels as described in Section 3.3. During backpropagation, we compute the gradient of the loss function with respect to each of the parameters starting from the parameters of the final layer as shown in Section 3.4. Finally, the gradients are used to update the parameters as shown in Section 3.5.

3.1 Dataset and Batches

Let the dataset contain N samples, where each sample is a pair of one input and the corresponding label. To train a network, we split the dataset into train, validation, and test sets. Typically, the split ratio is as follows

- Train set: 80% of the dataset
- Validation set: 10% of the dataset
- Test set: 10% of the dataset

During training, we select the best set of parameters by choosing the parameters that show the best performance on the validation set. We then report the final performance of the trained fully connected network on the test set. With this approach, we report final performance on data that has not been utilized during the training iterations.

Batches: In each iteration of the training, we randomly select B samples from the train set. We train using batches of data to efficiently use the computation resources (RAM, CPU) that are available to train the network. A batch of samples can be represented as a matrix $X_B \in \mathbb{R}^{d \times B}$, where each column in X_B corresponds to one data point in the batch.

3.2 Forward Pass

In the forward pass, the input is transformed into the output through a series of transformations corresponding to each of the layers in the fully connected network. In the first transformation, we transform the input $X_B \in \mathbb{R}^{d \times B}$ into

the output of the hidden layer $h_B \in \mathbb{R}^{d_h \times B}$ as shown in Equation 1, where $b_1 \in \mathbb{R}^{d_h \times 1}$ is the bias of the first layer.

$$h_B = W_1 \times X_B + b_1 \quad (1)$$

We introduce nonlinearity into fully connected networks by applying the ReLU activation function on the outputs of the hidden layers. In the two layered network shown in Figure 1, the ReLU activation function is applied as shown in Equation 2, where $\max()$ is an element-wise operation.

$$h_{Brelu} = \max(0, h_B) \quad (2)$$

The output of the activation function $h_{Brelu} \in \mathbb{R}^{d_h \times B}$ is transformed into output of the network $S \in \mathbb{R}^{n \times B}$ as shown in Equation 3, where $S \in \mathbb{R}^{n \times B}$ is the output score matrix, and $b_2 \in \mathbb{R}^{n \times 1}$ is the bias of the second layer.

$$S = W_2 \times h_{Brelu} + b_2 \quad (3)$$

After the forward pass, the network generates a score vector for each input sample. During inference, the predicted class corresponding to the input sample is determined by the class with the highest score. The size of the score vector is the same as the number of classes, and each element of the score vector corresponds to a class label.

3.3 Loss Computation

A loss function we can use for the classification task is the Cross Entropy Loss, which is shown in Equation 4, where $Y_j = [y_{1j}, y_{2j}, y_{3j}, \dots, y_{nj}]$ is the one-hot encoded vector (one-hot encoding is described in Appendix A) label of the data sample j , and $P_j = [p_{1j}, p_{2j}, p_{3j}, \dots, p_{nj}]$ is the probability distribution (the term *Probability Distribution* is described in Appendix C) of the network prediction over the n classes.

$$L_{CE} = -\frac{1}{B} \sum_{j=1}^B \sum_{i=1}^n y_{ij} \log p_{ij} \quad (4)$$

The score vector $S_j = [s_{1j}, s_{2j}, s_{3j}, \dots, s_{nj}]$ of each sample j is converted to its corresponding probability distribution P_j by applying the Softmax operation element-wise as shown in Equation 5, where $\exp(x) = e^x$.

$$p_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^n \exp(s_{kj})} \quad (5)$$

3.4 Backpropagation

In the backpropagation step, we compute the gradient of the loss function with respect to each of the parameters starting from the parameters of the last layer. The gradient of the loss function L_{CE} with respect to the probability distribution matrix is given by the Equations 6, 7, and 8. Derivation of the Equation 8 is shown in Appendix D.

$$P = \begin{bmatrix} P_{11} & P_{12} & P_{13} & \dots & P_{1B} \\ P_{21} & P_{22} & P_{23} & \dots & P_{2B} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ P_{n1} & P_{n2} & P_{n3} & \dots & P_{nB} \end{bmatrix} \quad (6)$$

$$\frac{dL_{CE}}{dP} = \begin{bmatrix} \frac{\partial L_{CE}}{\partial P_{11}} & \frac{\partial L_{CE}}{\partial P_{12}} & \frac{\partial L_{CE}}{\partial P_{13}} & \dots & \frac{\partial L_{CE}}{\partial P_{1B}} \\ \frac{\partial L_{CE}}{\partial P_{21}} & \frac{\partial L_{CE}}{\partial P_{22}} & \frac{\partial L_{CE}}{\partial P_{23}} & \dots & \frac{\partial L_{CE}}{\partial P_{2B}} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ \frac{\partial L_{CE}}{\partial P_{n1}} & \frac{\partial L_{CE}}{\partial P_{n2}} & \frac{\partial L_{CE}}{\partial P_{n3}} & \dots & \frac{\partial L_{CE}}{\partial P_{nB}} \end{bmatrix} \quad (7)$$

$$\frac{\partial L_{CE}}{\partial P_{ij}} = \begin{cases} -\frac{1}{B \times P_{ij}} & y_{ij} = 1 \\ 0 & y_{ij} = 0 \end{cases} \quad (8)$$

The gradient of the loss function L_{CE} with respect to the score matrix is given by Equations 9, 10, and 11. Derivation for Equation 11 is shown in Appendix E.

$$S = \begin{bmatrix} S_{11} & S_{12} & S_{13} & \dots & S_{1B} \\ S_{21} & S_{22} & S_{23} & \dots & S_{2B} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ S_{n1} & S_{n2} & S_{n3} & \dots & S_{nB} \end{bmatrix} \quad (9)$$

$$\frac{dL_{CE}}{dS} = \begin{bmatrix} \frac{\partial L_{CE}}{\partial S_{11}} & \frac{\partial L_{CE}}{\partial S_{12}} & \frac{\partial L_{CE}}{\partial S_{13}} & \dots & \frac{\partial L_{CE}}{\partial S_{1B}} \\ \frac{\partial L_{CE}}{\partial S_{21}} & \frac{\partial L_{CE}}{\partial S_{22}} & \frac{\partial L_{CE}}{\partial S_{23}} & \dots & \frac{\partial L_{CE}}{\partial S_{2B}} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ \frac{\partial L_{CE}}{\partial S_{n1}} & \frac{\partial L_{CE}}{\partial S_{n2}} & \frac{\partial L_{CE}}{\partial S_{n3}} & \dots & \frac{\partial L_{CE}}{\partial S_{nB}} \end{bmatrix} \quad (10)$$

$$\frac{\partial L_{CE}}{\partial S_{ij}} = \begin{cases} \frac{P_{ij}-1}{B} & y_{ij} = 1 \\ \frac{P_{ij}}{B} & y_{ij} = 0 \end{cases} \quad (11)$$

Propagating gradients back through the output layer, we get gradients of loss with respect to W_2 (Eq. 12), b_2 (Eq. 13), and h_{Brelu} (Eq. 14) as

$$\frac{dL_{CE}}{dW_2} = \frac{dL_{CE}}{dS} \times h_{Brelu}^T \quad (12)$$

$$\frac{dL_{CE}}{db_2} = \frac{dL_{CE}}{dS} \times \mathbf{1}_{b_2} \quad (13)$$

where $\mathbf{1}_{b_2} \in \{1\}^{B \times 1}$

$$\frac{dL_{CE}}{dh_{Brelu}} = W_2^T \times \frac{dL_{CE}}{dS} \quad (14)$$

Backpropagating gradients through the ReLU activation function, we get the gradient of Loss with respect to h_B as shown in Equation 15, where $A \cdot B$ represent element-wise multiplication between any compatible matrices A and B.

$$\frac{dL_{CE}}{dh_B} = \frac{dL_{CE}}{dh_{Brelu}} \cdot (h_B > 0) \quad (15)$$

Finally, backpropagating to parameters W_1 and b_1 , we get Equations 16 and 17 respectively.

$$\frac{dL_{CE}}{dW_1} = \frac{dL_{CE}}{dh_B} \times X^T \quad (16)$$

$$\frac{dL_{CE}}{db_1} = \frac{dL_{CE}}{dh_B} \times \mathbf{1}_{b_1} \quad (17)$$

where $\mathbf{1}_{b_1} \in \{1\}^{B \times 1}$

3.5 Parameter Updates

In each iteration of training, the update rules for the parameters are

$$W_1 = W_1 - \alpha \times \frac{dL_{CE}}{dW_1}$$

$$b_1 = b_1 - \alpha \times \frac{dL_{CE}}{db_1}$$

$$W_2 = W_2 - \alpha \times \frac{dL_{CE}}{dW_2}$$

$$b_2 = b_2 - \alpha \times \frac{dL_{CE}}{db_2}$$

where α is the learning rate.

A One-hot Encoding

In a typical dataset for the classification task, each data sample is given a label in the form of an index of one of the classes. For example, consider a dataset D with classes C_i where $i = 1, 2, 3, \dots, n$. If a data sample x belongs to class C_2 , then it will be given the label 2.

When we need to represent this label in the form of a probability distribution over n classes, we generate a one dimensional vector of length n where all the elements are zeros except for the element that corresponds to the class of the label which is one. In the above example, the one-hot encoded vector would be $[0, 1, 0, 0, \dots, 0]$ where the length of the vector is n .

B Rate of change and some common derivatives

B.1 Rate of change

Rate of change of a function $f(x)$ with respect to a variable x is shown in Equation 18

$$\text{rate of change of } f(x) \text{ w.r.t. } x = \frac{\Delta f(x)}{\Delta x} = \frac{f(x_2) - f(x_1)}{x_2 - x_1} \quad (18)$$

B.2 Derivatives

Equation 18 gives an accurate estimate of the rate of change when it is constant. However, to get an accurate estimate of the rate of change when it varies with x , we need to compute the rate of change with infinitesimally small change in x as shown in Equation 19

$$\lim_{\Delta x \rightarrow 0} \frac{\Delta f(x)}{\Delta x} = \frac{df(x)}{dx} \quad (19)$$

B.3 Common Derivatives

In this section, let's discuss some of the common derivatives that are useful to know.

B.3.1 Constant Function

Consider the function $f(x) = c$, where c is a constant value. The output of a constant remains constant even with changes in x . Since the input x does

not have an effect on the output of $f(x)$, the derivative of $f(x)$ w.r.t x is 0 as shown in Equation 20.

$$\frac{df(x)}{dx} = \frac{dc}{dx} = 0 \quad (20)$$

Exercise: Prove Equation 20. Hint: Substitute $f(x) = c$ in Equation 19.

B.3.2 Multiplication of two functions

Consider three functions, $f(x)$, $g(x)$, and $h(x)$ that are related by multiplication as $h(x) = f(x) \times g(x)$. The derivative of $h(x)$ can be expressed in terms of derivatives of $f(x)$ and $g(x)$ as shown in the Equation 21.

$$\frac{dh(x)}{dx} = f(x) \times \frac{d(g(x))}{dx} + g(x) \times \frac{df(x)}{dx} \quad (21)$$

B.3.3 Chain Rule

Consider three functions, $f(x)$, $g(x)$, and $h(x)$ that are related by composition as $h(x) = f(g(x))$. The derivative of $h(x)$ can be expressed in terms of derivatives of $f(x)$ and $g(x)$ as shown in the Equation 22.

$$\frac{dh(x)}{dx} = \frac{df(g(x))}{dx} = \frac{df(x)}{dg(x)} \times \frac{dg(x)}{dx} \quad (22)$$

B.3.4 Polynomial Function

Derivative of polynomial functions of the form $f(x) = x^n$ with respect to x is shown in the Equation 23.

$$\frac{d(f(x))}{dx} = \frac{dx^n}{dx} = nx^{n-1} \quad (23)$$

B.3.5 Exponential Function

Derivative of exponential functions of the form $f(x) = e^x$ with respect to x is shown in the Equation 24.

$$\frac{d(f(x))}{dx} = \frac{de^x}{dx} = e^x \quad (24)$$

B.3.6 Logarithmic Function

Derivative of logarithmic functions of the form $f(x) = \log x$ with respect to x is shown in the Equation 25.

$$\frac{d(f(x))}{dx} = \frac{d \log x}{dx} = \frac{1}{x} \quad (25)$$

B.4 Partial Derivatives

Consider a function with multiple variables, also called as a multivariate function. If we want to compute derivate of such a function with respect to any one of the variables, then we compute the derivate of that function with respect to that variable while treating the other variables as constants.

For example, consider the multivariate function $f(x, y) = 3x^2y$, where x and y are variables. The partial derivatives of the function $f(x, y)$ are shown in the Equations 26 and 27 respectively.

$$\frac{\partial f(x, y)}{\partial x} = 6xy \quad (26)$$

$$\frac{\partial f(x, y)}{\partial y} = 3x^2 \quad (27)$$

C Probability Distributions

Probability distribution refers to a set of probabilities, each of which is a value between 0 and 1 that represents the possibility of the occurrence of the corresponding event. Consider a set of possible events $\{E_1, E_2, E_3, \dots, E_n\}$ with probabilities $\{p_1, p_2, p_3, \dots, p_n\}$ respectively. For the vector $[p_1, p_2, p_3, \dots, p_n]$ to be a probability distribution, the conditions shown in Equations 28 and 29 should be satisfied.

$$p_i \in [0, 1], \text{ where } i \in \{1, 2, 3, \dots, n\} \quad (28)$$

$$\sum_{i=1}^n p_i = 1 \quad (29)$$

In the case of classifying a data point, each event E_i can be characterized as the event in which the data points belongs to class C_i , and the probability p_i represents the chance of the data point belonging to the class C_i . When we want to predict the class of a data point using a deep learning model, we can give the data point as input to the deep learning model to generate a

probability distribution over the possible classes, and assign the data point to a class with the highest probability.

For example, consider a deep learning model M that has been trained to classify its inputs into one of the classes C_1 , C_2 , and C_3 . If M generates the probability distribution $[0.2, 0.5, 0.3]$ for an input data point x , then we say that the class predicted by M for the data point x is C_2 .

D Derivative of the Cross Entropy Loss

Consider a data point (X, c) , where X is the input data and c is the label. Let the corresponding one-hot encoded vector be $Y = [y_1, y_2, \dots, y_c, \dots, y_{n-1}, y_n]$. The values of the elements of Y are shown in the Equation 30.

$$y_i = \begin{cases} 1 & i = c \\ 0 & i \neq c \end{cases} \quad (30)$$

Let the probability distribution vector generated by a classification model with input X be $P = [p_1, p_2, \dots, p_c, \dots, p_{n-1}, p_n]$. The cross entropy loss computation for this data sample is shown in the Equation 31.

$$L_{CE} = - \sum_{i=1}^n y_i \log p_i = -y_1 \log p_1 - y_2 \log p_2 - \dots - y_c \log p_c - \dots - y_n \log p_n \quad (31)$$

Substituting 30 in 31, we get

$$\begin{aligned} L_{CE} &= -0 \log p_1 - 0 \log p_2 - \dots - 1 \log p_c - \dots - 0 \log p_n \\ &= -\log p_c \end{aligned} \quad (32)$$

Since L_{CE} has only one variable p_c , its derivative $\frac{\partial L_{CE}}{\partial p_i} = 0$ if $i \neq c$. When $i = c$, $\frac{\partial L_{CE}}{\partial p_i} = \frac{\partial(-\log p_c)}{\partial p_c} = \frac{-1}{p_c}$. Combining the two cases, we get the derivative of the cross entropy loss as shown in the Equation 33.

$$\frac{\partial L_{CE}}{\partial p_i} = \begin{cases} -\frac{1}{p_i} & i = c \\ 0 & i \neq c \end{cases} \quad (33)$$

E Backpropagation through Softmax

Consider the input data X , one-hot encoded label vector Y , and the probability distribution vector P as described in Appendix D. Let the score vector generated by the model with input X be $S = [s_1, s_2, \dots, s_c, \dots, s_n]$. Since the

probability distribution P is computed using the softmax function with S as input, each element of P is shown in the Equation 34, where $i = 1, 2, \dots, n$.

$$p_i = \frac{\exp(s_i)}{\sum_{j=1}^n \exp(s_j)} \quad (34)$$

From Equation 32, we can see that the loss function L_{CE} is a function of the variable p_c , where c is the ground truth label of the input data X . From 34, we can see that $p_c = \frac{\exp(s_c)}{\sum_{j=1}^n \exp(s_j)}$. Thus, p_c is a function of every element s_i , where $i = 1, 2, \dots, n$. Since L_{CE} is a function of p_c and p_c is a function of $s_i \forall i$, we can apply the chain rule (Appendix B.3.3) as shown in the Equation 35.

$$\frac{\partial L_{CE}}{\partial s_i} = \frac{\partial L_{CE}}{\partial p_c} \times \frac{\partial p_c}{\partial s_i}; \text{ where } i = 1, 2, \dots, n \quad (35)$$

From the Equation 33, we know that $\frac{\partial L_{CE}}{\partial p_c} = -\frac{1}{p_c}$. The unknown term left in the Equation 35 is $\frac{\partial p_c}{\partial s_i}$. We can compute $\frac{\partial p_c}{\partial s_i}$ based on the Equation 34 as

$$\begin{aligned} \frac{\partial p_c}{\partial s_i} &= \frac{\partial \left(\frac{\exp(s_c)}{\sum_{j=1}^n \exp(s_j)} \right)}{\partial s_i} \\ &= \frac{\partial (\exp(s_c) \times \frac{1}{\sum_{j=1}^n \exp(s_j)})}{\partial s_i} \end{aligned} \quad (36)$$

Expanding the R.H.S of the Equation 36 by applying the product rule of derivatives (Appendix B.3.2), we get

$$\frac{\partial p_c}{\partial s_i} = \exp(s_c) \times \frac{\partial \left(\frac{1}{\sum_{j=1}^n \exp(s_j)} \right)}{\partial s_i} + \frac{1}{\sum_{j=1}^n \exp(s_j)} \times \frac{\partial \exp(s_c)}{\partial s_i} \quad (37)$$

Computing each of the partial derivatives on the R.H.S of the Equation 37 separately in Equations 38 and 39.

$$\begin{aligned} \frac{\partial \left(\frac{1}{\sum_{j=1}^n \exp(s_j)} \right)}{\partial s_i} &= -\frac{1}{(\sum_{j=1}^n \exp(s_j))^2} \times \frac{\partial \sum_{j=1}^n \exp(s_j)}{\partial s_i} \\ &= -\frac{1}{(\sum_{j=1}^n \exp(s_j))^2} \times \exp(s_i) \\ &= -\frac{\exp(s_i)}{(\sum_{j=1}^n \exp(s_j))^2} \end{aligned} \quad (38)$$

$$\begin{aligned}
\frac{\partial \exp(s_c)}{\partial s_i} &= \exp(s_c) \times \frac{\partial s_c}{\partial s_i} \\
&= \begin{cases} \exp(s_c) \times 1 & i = c \\ \exp(s_c) \times 0 & i \neq c \end{cases} \\
&= \begin{cases} \exp(s_c) & i = c \\ 0 & i \neq c \end{cases}
\end{aligned} \tag{39}$$

Substituting Equation 38 in Equation 37, we get

$$\begin{aligned}
\frac{\partial p_c}{\partial s_i} &= \exp(s_c) \times \left(-\frac{\exp(s_i)}{(\sum_{j=1}^n \exp(s_j))^2} + \frac{1}{\sum_{j=1}^n \exp(s_j)} \right) \times \frac{\partial \exp(s_c)}{\partial s_i} \\
&= -\frac{\exp(s_c)}{\sum_{j=1}^n \exp(s_j)} \times \frac{\exp(s_i)}{\sum_{j=1}^n \exp(s_j)} + \frac{1}{\sum_{j=1}^n \exp(s_j)} \times \frac{\partial \exp(s_c)}{\partial s_i} \\
&= -p_c \times p_i + \frac{1}{\sum_{j=1}^n \exp(s_j)} \times \frac{\partial \exp(s_c)}{\partial s_i}
\end{aligned} \tag{40}$$

Substituting Equation 39 in Equation 40, we get

$$\begin{aligned}
\frac{\partial p_c}{\partial s_i} &= \begin{cases} -p_c \times p_c + \left(\frac{1}{\sum_{j=1}^n \exp(s_j)} \right) \times \exp(s_c) & i = c \\ -p_c \times p_i + \frac{1}{\sum_{j=1}^n \exp(s_j)} \times 0 & i \neq c \end{cases} \\
&= \begin{cases} -p_c \times p_c + \left(\frac{\exp(s_c)}{\sum_{j=1}^n \exp(s_j)} \right) & i = c \\ -p_c \times p_i & i \neq c \end{cases} \\
&= \begin{cases} -p_c \times p_c + p_c & i = c \\ -p_c \times p_i & i \neq c \end{cases} \\
&= \begin{cases} -p_c \times (p_c - 1) & i = c \\ -p_c \times p_i & i \neq c \end{cases}
\end{aligned} \tag{41}$$

Substituting Equations 33 and 41 in Equation 35, we get

$$\begin{aligned}
\frac{\partial L_{CE}}{\partial s_i} &= \begin{cases} \frac{\partial L_{CE}}{\partial p_c} \times (-p_c \times (p_c - 1)) & i = c \\ \frac{\partial L_{CE}}{\partial p_c} \times (-p_c \times p_i) & i \neq c \end{cases} \\
&= \begin{cases} -\frac{1}{p_c} \times (-p_c \times (p_c - 1)) & i = c \\ -\frac{1}{p_c} \times (-p_c \times p_i) & i \neq c \end{cases} \\
&= \begin{cases} p_c - 1 & i = c \\ p_i & i \neq c \end{cases}
\end{aligned} \tag{42}$$